RightSize Bench: Learning Kubernetes Resource Requests from Code

Sizing the pod before it runs, through a raw Job or an AI-native interface

Niels Bantilan
Union.ai
niels@union.ai

November 2026

Abstract

Before a batch pod runs on Kubernetes, someone has to decide how much memory and CPU it gets, and whether it gets a GPU. Too little makes it fail or run slowly, and too much wastes the cluster. We present RightSize Bench, which scores approaches to this decision by the dollars it takes to get each run to finish, compared with the cheapest request that would have worked. The benchmark contains 32,082 measured runs of 33 families of generated Python tasks that use real libraries, from dataframes and SQL to PyTorch training and sharded language models. Each run appears twice, once as a raw Kubernetes Job, with its inputs staged in one of four ways, and once as a task submitted through Flyte, an AI-native interface to Kubernetes that adds typed I/O, caching, retries and durable execution at a measured cost of about 70 MiB per pod. We validated the labels in about 480 real pods, which exposed and fixed two errors in our first release.

We compare static requests, two kinds of rules, gradient-boosted trees, a zero-shot 27B model, a fine-tuned 1.7B decision model further trained with reinforcement learning against the cost, opinion pools that combine the trees with each language model, and a pre-registered novelty gate, over three independent trainings. On new tasks from familiar families, the pool of trees and the decision model has 6.6% mean regret on Job pods [5.6, 7.6] (dollar regret 27.6%). On held-out families, the RL model and a rule that scales by input size and reads the manifest tie at 39%. When an out-of-memory failure costs as much as resubmitting a Job by hand, that rule is the best approach on new code. Models trained on Jobs read the manifest: when only the volume medium changes, they move their request the right way about three times in four. Adding Flyte data does not improve them on Jobs, and a model trained only on Flyte tasks transfers poorly to Jobs.

1 Introduction

Every container that runs on Kubernetes asks for resources before it starts [13]. The numbers matter in different ways. Memory is a hard limit, so a container that uses more than it asked for is killed and has to be run again. CPU is a soft limit, so a container that wants more cores than it was given is throttled and finishes late. A GPU is a choice of machine: if it is too small the task runs out of device memory, if it is missing a GPU task fails at start, and if a CPU task gets one it pays for hardware it never uses. In practice engineers pick these numbers by hand, usually once and usually generously, and large clusters run well below what their jobs reserve [21, 22].

Autoscalers that learn from usage history [15, 18] work well for long-running services. Batch work is harder. The first run of a batch task has no history, and later runs often see inputs of a different size. Still, much of what decides the right request is known before the pod starts. A scheduler can read the code the pod

will run, the size and format of its inputs, the way the pod spec stages those inputs, and what earlier runs of the same task used. This paper asks how well different approaches turn that information into a request, and measures the answer in dollars.

We treat the Kubernetes pod as the unit of work throughout. Teams submit pods in different ways. The most direct is a `batch/v1` Job, which pairs a container image with a manifest. Another is to write a typed Python function and let an orchestrator build the pod. We use Flyte [5] as the example of the second route. Flyte acts as an AI-native interface to Kubernetes: it turns a function into a pod, and adds typed inputs and outputs, caching, retries and durable execution along the way. In both cases the same user code runs in the same kind of pod. The differences are at the edges of the pod, in how inputs arrive, what else shares the container, and what the monitoring history reports. For Flyte, those conveniences cost 55–70 MiB of runtime memory in the task's container, which we measure directly (Section 7).

This paper makes five contributions.

1. We build a benchmark of the whole pod request from 32,082 measured runs of 33 families of generated Python tasks, covering dataframes, SQL, linear algebra, classical ML, PyTorch training and inference, and sharded language models. Each run has measured peak memory, CPU use at the 15-second resolution of cluster monitoring, and peak GPU memory across seven machine shapes (Section 5).
2. We present every run both as a raw Kubernetes Job and as a Flyte task. The Job stages its inputs in one of four ways, and its labels follow from an explicit accounting of what differs between the two pods (Section 6).
3. We check that accounting in about 480 real pods, reading each container's own cgroup counters (Section 7). The check found two defects in our first release, both now fixed. The Flyte runtime constant was 2.6 times too large, and some init containers were too small to hold the inputs they staged. It also showed that the standard `ru_maxrss` measurement leaves about a third of labels one cell higher than necessary.
4. We score requests with a single cost model that covers memory, CPU and GPU together, and report both mean and dollar-weighted regret with task-clustered confidence intervals (Section 8).
5. We compare eleven kinds of approach over three independent trainings, from static requests and hand-written rules to gradient-boosted trees, a zero-shot 27B model, a fine-tuned 1.7B decision model trained further with reinforcement learning, opinion pools, and a pre-registered novelty gate (Sections 9 and 11). Two controlled tests ask what the models actually read. One changes only the manifest, and the other removes the task's name from its code.

2 Background: one pod, two ways to submit it

Requests, limits and cgroups. Each Kubernetes container declares a memory and CPU *request*, which the scheduler reserves on a node, and a *limit*, which the kubelet enforces through the container's cgroup [9, 13]. Under cgroup v2, a container whose charged memory reaches its limit is OOM-killed. Both anonymous memory and shared memory (`shmem`) count toward that limit, while clean page cache does not, because the kernel can reclaim it first. A CPU limit works differently. It is enforced as a CFS quota, so a container that exceeds it is slowed down rather than killed. Accelerators are requested as whole devices on node pools that carry them. In this paper a request also sets the limit, which is the usual choice for batch pods because it makes scheduling and failures predictable.

How the pod spec changes a task's memory. Three details of a pod spec can change how much memory a batch task needs without any change to its code. The first is the medium of the volume that holds its inputs. An `emptyDir` on node disk keeps files as page cache, which the kernel reclaims under pressure. An `emptyDir` with `medium: Memory` is a tmpfs, and every byte written to it is charged to memory as `shmem` [14]. Staging a 2 GiB input on tmpfs therefore adds 2 GiB to the pod's need. The second detail is who fetches the inputs. An

init container runs to completion before the main container starts and has its own limit, whereas a download inside the job runs in the job's container, so the client library's memory and threads count against the job. The third is anything else that shares the container, such as an orchestration runtime or a client library.

There is one subtlety with tmpfs. A page is charged to the cgroup of the container that writes it, so pages written by an init container are not charged to the main container. When the init container exits, though, its pages move to the pod's cgroup and still count against the pod's overall limit. We confirm this in real pods in Section 7.

Raw Jobs. A `batch/v1` Job runs a pod template until it succeeds. Its `backoffLimit` says how many failed pods it will replace before giving up. Batch teams often set it to zero and resubmit failures by hand or from an outer scheduler. A Job has to stage its own inputs, either with an init container or in the job's code.

Flyte as an interface to Kubernetes. Flyte [5] turns a typed Python function into a Kubernetes pod. A user writes `@env.task async def f(x: File) -> dict`, and Flyte builds the image, renders the pod spec, moves typed inputs and outputs through object storage, caches results by their inputs, retries failed attempts, and records each execution so that a long pipeline can resume after a node is lost. None of this changes what the function does inside the container. It changes the pod's edges instead. Inputs arrive through Flyte's own download, the container also runs Flyte's runtime, and the execution history reports the container's peak with that runtime included. We measure the runtime at 55 MiB at rest, and a median 70 MiB above the same workload run as a plain Job (Section 7).

For resource sizing, then, Flyte is a convenience layer over the same pod. It costs a little memory in exchange for typed I/O, caching, retries and durable execution. An approach that can size a Kubernetes pod should be able to size a Flyte task using the same knowledge plus the runtime.

3 Related work

Sizing from history. Autopilot [18] sets Google's container limits from the usage history of a job's tasks, using moving-window and ML recommenders. Kubernetes' Vertical Pod Autoscaler [15] recommends requests from a decaying histogram of observed usage. Borg traces show that most jobs over-request [21, 22]. Resource Central [1] predicts VM utilization from a subscription's history. All of these size workloads that have run before, and run long enough to observe. RightSize Bench reports first runs (no history) and runs with history separately. Quasar [2] classifies new workloads from short profiling runs; RightSize Bench allows no profiling, only what a scheduler can read before the pod starts.

Learned predictors. Quantile regression [12] with gradient-boosted trees [6, 11] is a standard way to predict a distribution of resource need, and is the strongest classical approach here. Our decision model replaces a language model's output layer with a pointer head [23] that scores option tokens written in the prompt. It is adapted with LoRA [10] on Qwen3 [17].

Training on the objective. Group-relative policy optimization (GRPO) [19] drops PPO's value network by normalizing rewards within a group of samples for the same prompt. Our policy is three categorical distributions, so a group is sampled from one forward pass, and the reward is the benchmark's own cost simulator.

Combining and routing models. Logarithmic opinion pools [7] combine calibrated distributions by a weighted geometric mean. Temperature scaling [8] calibrates each model before pooling. Distance to the k

nearest training points is a standard out-of-distribution signal [20]; our gate uses it to choose between two models.

4 Problem

A *run* is one execution of a task on one set of inputs, in one pod. Before the pod starts, an approach may look at three things. The first is the task's code as it was submitted: for a raw Job that means `job.py` and the Job manifest, and for a Flyte task it means the task module (Section 6). The second is the metadata a scheduler can read about this run's inputs, namely their sizes and formats and whatever the format's header exposes, such as the row count and schema of a Parquet file or the shape and dtype of a `.npy` array. The third is what earlier runs of the same task used: their peak memory, peak and mean cores, peak GPU memory and utilization, wall time and input sizes. These are reported the way the platform's monitoring would show them, which means the container's own peak for a Job and the peak including the runtime for a Flyte task.

An approach never sees the task's family, the hidden settings it was generated with, which corpus it came from, or anything measured on the run itself. A unit test enforces this for every feature and every field of the prompt. From what it can see, the approach chooses a request on three axes (Table 1). A GPU request lands on the cheapest machine that can hold it (Table 2). On a GPU node the host memory and cores come with the instance, so they are not scored.

Table 1: The request grid.

axis	cells	values
memory	17	256, 384, 512, 768 Mi; 1, 1.5, 2, 3, 4, 6, 8, 12, 16, 24, 32, 48, 64 Gi (half powers of two)
CPU	16	0.25, 0.5, 0.75, 1, 1.25, 1.5, 1.75, 2, 2.5, 3, 3.5, 4, 5, 6, 7, 8 cores
GPU	18	none, or peak GPU memory 1, 1.5, 2, 3, 4, 6, 8, 10, 12, 15, 22, 32, 44, 64, 88, 120, 176 GiB

Table 2: The GPU machine ladder: the cluster's GPU node pools, with on-demand `us-east-1` prices for the whole node. Usable GiB is NVML's total, rounded down. The A10G is never the cheapest shape for any cell.

shape	instance	GPUs	usable GiB	$/h
T4	g4dn.xlarge	1	15	0.526
L4	g6.2xlarge	1	22	0.978
A10G	g5.2xlarge	1	22	1.212
L40S	g6e.2xlarge	1	44	2.242
4×L4	g6.12xlarge	4	4×22	4.602
8×T4	g4dn.metal	8	8×15	7.824
4×L40S	g6e.12xlarge	4	4×44	10.493

5 Dataset

The benchmark combines three corpora. Each one was built by its own single-axis benchmark using the same recipe. We wrote families of code generators, sampled tasks from each family, and measured several runs of each task at different input sizes in Kubernetes pods. Table 3 summarizes the three corpora, and Appendix A lists every family. All of these measurements describe the workload itself, independent of how its pod was submitted. Section 6 then works out what a Job pod and a Flyte pod each need, starting from the same measurement.

5.1 Families, tasks and runs

A *family* is a Python generator. Given a seed, it writes a task function and the specification of its inputs, and it draws a few hidden settings, which we call knobs, that decide how input bytes turn into resource use. A pandas family, for example, decides whether to project columns, which dtype backend to use and whether to downcast floats. A training family picks width, depth, batch size, mixed precision and checkpointing, and a CPU family picks a thread-pool size or leaves it unset. The knobs show up only in the code an approach reads. They are never given to an approach directly, and we keep them only to slice the results.

Each run scales the leading dimension of the task's inputs by a factor drawn log-uniformly from $[0.25, 4]$ for RightSizeMem and from $[0.5, 2]$ for the CPU and GPU corpora. Runs are shuffled, so a task's history can be larger or smaller than the current run. The inputs are synthetic: normal floats, uniform integers, dictionary-encoded categories, random strings, Zipf-distributed word ids for text, and random bytes for images, written as Parquet, CSV, JSONL or `.npy`. CPU and GPU workloads loop until a per-task time budget runs out (30–60 s for CPU and 45–90 s for GPU), while RightSizeMem workloads run to completion.

Table 3: The three corpora. RightSizeGPU's 2,052 unusable runs: 1,569 censored (out of memory on the largest shape they reached), 51 failed, and 432 the combined bench drops because their peak filled the device, which makes the measured peak only a lower bound. RightSizeMem's 166 unusable runs exceeded its 20 GiB guard or 900 s timeout.

corpus	release	families	held out	tasks/fam.	runs/task	measured	usable	labels
RightSizeMem	v2	12	2	400	3	14,400	14,234	peak memory; mean cores
RightSizeCPU	big-1	11	2	450	2	9,900	9,900	peak memory; busiest 15 s
RightSizeGPU	big-2	10	3	500	2	10,000	7,948	peak GPU memory, sharding
combined	k8s-2	33	7			34,300	32,082	

5.2 RightSizeMem v2: peak memory

RightSizeMem has twelve families, chosen because they turn input bytes into memory in different ways. They include pandas group-bys and joins, polars pipelines in eager, lazy and streaming modes, DuckDB queries with and without a memory limit, chunked CSV and JSONL readers, NumPy linear algebra with optional memory mapping, scikit-learn estimators, TF-IDF, a small PyTorch model on CPU, image batches and pure-Python grouping.

Each run uses three processes. A child process writes the inputs to a temporary directory, a fresh interpreter runs the task body on those files, and the parent reads the child's `ru_maxrss` and CPU time from `wait4`. BLAS and OpenMP are limited to one thread, and workloads set their own threads in code. The workload's peak memory w is its `ru_maxrss`. As we show in Section 7, on Linux this value includes the measuring parent's own peak at the time of `exec`, about 158 MiB here, which makes about a third of the smaller workloads' labels one cell higher than needed.

5.3 RightSizeCPU big-1: the busiest 15 seconds

RightSizeCPU has eleven families, chosen for how they use cores, since CPU need depends on parallelism much more than on input size. They cover NumPy BLAS with and without a thread cap, polars and DuckDB with explicit thread settings, scikit-learn ensembles, serial pandas, Python thread pools that hold or release the GIL, process pools, a CPU PyTorch model, pyarrow CSV parsing, an I/O-bound pool and a task with two phases.

CPU time for the whole process tree is sampled every second. Each pod runs two measurements side by side on disjoint sets of seven cores, and a check of 44 runs against single-run pods put all 44 within 25% of each other. The CPU label is the busiest 15-second average, because that is the resolution at which cluster monitoring (Prometheus scraping cAdvisor every 15 s) sees a container. Peak memory is measured as in RightSizeMem.

5.4 RightSizeGPU big-2: peak GPU memory across a machine ladder

RightSizeGPU has ten PyTorch families: a tabular MLP, a CNN, a CNN limited by its data loader, a GPT language model, a transformer encoder, a recommender with sharded embeddings, blocked matrix multiplication, nearest-neighbour search, fine-tuning of decoders from 0.6B to 7B parameters split across GPUs, and generation with a KV cache. NVML reports memory and utilization on every visible GPU every half second. Each run starts on a T4 and moves up the machine ladder (Table 2) whenever it runs out of device memory, and a run that runs out of memory on the largest machine it reached is censored. The oracle machine for a run is the cheapest one it fits on. For machines a run was not measured on, we model the fit from its per-GPU peak and, for sharded code, its imbalance across GPUs.

5.5 Splits and history

Runs are split by task, never by run. A hash of the task id sends 70% of each family's tasks to training, 10% to validation and 20% to test. The seven held-out families go entirely to a separate *unseen* split. The test split therefore measures how well an approach handles new tasks from familiar families, and the unseen split measures how well it handles new kinds of code. Every setting that an approach tunes is chosen on validation, which contains familiar families only.

The k-th run of a task carries the task's earlier runs as its history. A task's first run has no history at all, and these cold starts make up 33–50% of each split.

Table 4: Usable runs per split. Each run appears twice in the bench, once as a Job and once as a Flyte task, in the same split.

source	train	val	test	unseen	unseen families
mem	8,457	1,138	2,316	2,323	duckdb_sql, image_batch
cpu	5,628	838	1,634	1,800	duckdb_sql, pyarrow_csv
gpu	3,968	542	1,145	2,293	recsys_train, knn_search, llm_generate
runs	18,053	2,518	5,095	6,416	
pods (rows)	36,106	5,036	10,190	12,832	

6 Every run as a Job and as a Flyte task

The workload code was generated as Flyte tasks, but nothing in the body of a task depends on Flyte. We use that to put every measured run into the benchmark twice, once as a raw Kubernetes Job and once as a Flyte task, with the task body unchanged in both. To make the Job version, `job.py`, we turn `File` parameters into paths, drop the `flyte` import, and add a `__main__` block that stages the inputs, calls the function and writes `result.json` for upload. A `batch/v1` manifest goes with it, carrying the image, the input locations, the thread settings the measurement used, the staging volume and an empty `resources` block, which is the request still to be decided. Figure 1 shows one task written both ways.

6.1 Stagings

How a Job's inputs reach its container is the biggest thing that changes its memory need. Each task is assigned one of four stagings by a hash of its id, and all of its runs share it (Table 5). Inputs are fetched either by an init container running `aws-cli` or by the job itself with boto3, and they land either on node disk or on a memory-backed volume.

6.2 Label accounting

Let w be the workload's measured peak memory (Section 5) and let B be the size of the staged inputs plus the result file, with each file rounded up to whole pages. The two kinds of pod need

$$\begin{aligned} \text{Job:} \quad & 1.05\,\big(w + 48\,\text{MiB} \cdot [\text{boto3}] + B \cdot [\text{tmpfs}]\big), \\ \text{Flyte:} \quad & 1.05\,\big(w + R\big), \qquad R = 70\,\text{MiB}, \end{aligned}$$

where the factor 1.05 is headroom and the memory label is the smallest grid cell at or above the need. Each term has a simple reason. Files on a memory-backed volume are charged to memory byte for byte, while the same files on node disk are reclaimable page cache and add nothing, since the measured workloads already read local files. The boto3 client adds 48 MiB to a workload's peak. Its download also runs inside the job, taking d = bytes/440 MB/s + 0.4 s per object + 0.7 s at about 1.8 cores, so we prepend that window to the

Table 5: The four Job stagings.

staging	share	who fetches the inputs	where they land	memory need beyond the workload
init-disk	30%	aws-cli init container	`emptyDir` on node disk	—
init-tmpfs	20%	aws-cli init container	`emptyDir`, `medium: Memory`	+B (staged bytes)
boto3-disk	30%	the job itself, boto3	`emptyDir` on node disk	+48 MiB (boto3)
boto3-tmpfs	20%	the job itself, boto3	`emptyDir`, `medium: Memory`	+48 MiB +B

Flyte task

```
import flyte
from flyte.io import File
import pandas as pd

env = flyte.TaskEnvironment(name="pandas-join",
     image=flyte.Image.from_debian_base().
     with_pip_packages("pandas", "pyarrow"))


@env.task
async def join_orders_catalog(orders: File, catalog:
      File) -> dict:
    orders_path = await orders.download()
    catalog_path = await catalog.download()
    orders = pd.read_parquet(orders_path)
    catalog = pd.read_parquet(catalog_path)
    joined = orders.merge(catalog, on="sku", how="
     inner")
    joined["revenue"] = joined["qty"] * joined["
     price"]
    by_brand = joined.groupby("brand", observed=True
     )["revenue"].sum()
    return {"rows": int(len(joined)), "brands": int(
     len(by_brand))}
```

Job manifest (boto3-tmpfs)

```
apiVersion: batch/v1
kind: Job
metadata:
  generateName: pandas-join-
  labels: {app.kubernetes.io/name: pandas-join}
spec:
  backoffLimit: 0
  template:
    spec:
      restartPolicy: Never
      containers:
        - name: main
          image: registry.internal/batch/pandas-join
    :3ee28a49
          command: [python, /app/job.py]
          env:
            - {name: INPUT_ORDERS, value: "s3://
    batch-data/pandas-join/a1b2c3d4/run-1/orders.
    parquet"}
            - {name: INPUT_CATALOG, value: "s3://
    batch-data/pandas-join/a1b2c3d4/run-1/catalog.
    parquet"}
            - {name: OMP_NUM_THREADS, value: "1"}
            - {name: OPENBLAS_NUM_THREADS, value:
    "1"}
            - {name: MKL_NUM_THREADS, value: "1"}
            - {name: OUTPUT_URI, value: "s3://batch-
    data/pandas-join/a1b2c3d4/run-1/out/"}
          resources: {}  # to be decided
          volumeMounts: [{name: data, mountPath: /
    data}]
      volumes:
        - name: data
          emptyDir: {medium: Memory}
```

`job.py` (boto3 staging)

```
import pandas as pd
import json
import os
import boto3


def join_orders_catalog(orders_path: str, catalog_path:
      str) -> dict:
    orders = pd.read_parquet(orders_path)
    catalog = pd.read_parquet(catalog_path)
    joined = orders.merge(catalog, on="sku", how="inner
     ")
    joined["revenue"] = joined["qty"] * joined["price"
     ]
    by_brand = joined.groupby("brand", observed=True)[
     "revenue"].sum()
    return {"rows": int(len(joined)), "brands": int(
     len(by_brand))}


def fetch(uri: str) -> str:
    bucket, key = uri.removeprefix("s3://").split("/",
      1)
    path = os.path.join("/data", os.path.basename(key))

    s3.download_file(bucket, key, path)
    return path


if __name__ == "__main__":
    s3 = boto3.client("s3")
    result = join_orders_catalog(fetch(os.environ["
     INPUT_ORDERS"]), fetch(os.environ["INPUT_CATALOG"
     ]))
    os.makedirs("/data/out", exist_ok=True)
    with open("/data/out/result.json", "w") as f:
        json.dump(result, f)
    bucket, key = os.environ["OUTPUT_URI"].removeprefix
     ("s3://").split("/", 1)
    s3.upload_file("/data/out/result.json", bucket, key
      + "result.json")
```

Figure 1: One RightSizeMem task, as a Flyte task and as a raw Job. The disk staging differs from this manifest in one line, `emptyDir: {}`. The init-container stagings add an `aws-cli` init container that copies the inputs to `/data`, and `job.py` then reads them from `argv` (Appendix B).

run's CPU series. For a light job with a large input, the download can be the busiest 15 seconds of the run. The Flyte runtime R is 70 MiB, measured in real task pods (Section 7). Earlier releases used a value of 180 MiB that had never been measured. Staging does not change GPU need.

The manifest sizes the init container at 256 MiB + B for tmpfs stagings, so that it can hold what it writes. Our first release used a flat 256 Mi, which real pods showed cannot work for larger inputs (Section 7). Histories are reported in each interface's own units: a Job's earlier runs report the container's peak as cAdvisor would, and a Flyte task's report the peak including the runtime, as Flyte's UI would.

The staging constants come from an in-cluster probe using boto3 1.43 under cgroup v2. A bare interpreter holds 11 MiB. With boto3 imported, a client built and a file downloaded, the process holds 74–80 MiB, and it never peaks above that final level. Inside a real workload boto3 adds only 48 MiB at the median, because it shares modules the workload has already loaded. Download throughput rises from 122 MB/s for a 64 MiB object to 417–431 MB/s for a 4 GiB one.

7 Checking the labels in real pods

Because the labels in Section 6.2 are derived rather than measured directly, we checked them twice. The first check (rt-k8sval-2) re-ran 176 corpus runs both as Flyte tasks and as their rendered `job.py`, as sibling processes inside one pod. It confirmed the arithmetic of the accounting. Measured over predicted memory was 0.999–1.000 at the median for every staging, and tmpfs was charged at exactly the input size. A single pod has a single memory cgroup, though, so this check could not show where the kernel charges pages when a pod has several containers.

The second check (rt-k8spod-1) used real pods. We took 100 runs (48 from RightSizeMem, 44 from RightSizeCPU, and 8 init-tmpfs runs with inputs over 300 MiB) and gave each of them pods of its own, about 480 in all (Table 6). Every container sampled its own cgroup's memory counters every 50 milliseconds.

The Flyte runtime. Before a task's own module is imported, the Flyte task container already holds 55.0 MiB of anonymous memory, with almost no variation across the 100 runs (p10–p90 54.9–55.0 MiB). Its peak is a median 70 MiB above the same run's peak as a plain Job (p10 53, p90 242 MiB), so running a workload inside the runtime's process costs slightly more than the runtime does on its own. The 180 MiB used in earlier releases had no measurement behind it. With that value, the Flyte label matched the real container's cell for only 23% of runs and was one cell too high for 70%. With 70 MiB, it matches for 58%, is one cell high for 33% and one cell low for 9%. The low ones are process pools, where the container counts every worker process, and some DuckDB and polars runs. The k8s-2 release uses 70 MiB, which lowers 61% of the Flyte labels on CPU nodes by one cell.

Table 6: The pods in the real-pod check (rt-k8spod-1).

pod	what it measures
Flyte task	the run as a real Flyte task: the rendered module in a fresh task pod, with its inputs downloaded by Flyte
Job, generous limit	the pod the manifest describes, with each container reading its own cgroup
Job at its label	does the run fit when the job container's limit is set to the label?
Job one cell lower	does the cell below the label fail?
init-tmpfs extras	the first release's flat 256 Mi init container, and the job at its label minus the staged bytes

Table 7: Job pods with the limit at the label cell and one cell below it (rt-k8spod-1). "Other" is a failure that was not an OOM kill.

	at label			one cell below		
staging	ran	OOM	other	ran	OOM	other
init-disk	22	1	0	17	5	1
init-tmpfs	30	1	0	24	7	0
boto3-disk	23	0	0	16	6	1
boto3-tmpfs	23	0	0	15	7	1

Where tmpfs pages are charged. A tmpfs page is charged to the container that writes it. When the job downloads its own inputs (boto3-tmpfs), the job container's shared memory grows by the staged size times 1.0001 at the median ($n = 23$). When an init container stages them (init-tmpfs), the init container's shared memory equals the staged size and the job container's stays at zero ($n = 31$). The pages outlive the init container, however, and still count against the pod. With its limit set to the label minus the staged bytes, the job was OOM-killed in 9 of 31 init-tmpfs pods, and with the full label 30 of 31 ran. So adding the staged bytes to the label is correct, even though the pages are enforced through the pod's limit rather than the container's.

The first release's init containers. With the flat 256 Mi init container of the first release, every staging up to 162 MiB ran and none of 254 MiB or more did. Ten of those were confirmed OOM-killed while staging, and six more failed. In that release, 982 of the 2,756 evaluated init-tmpfs Jobs staged more than 200 MiB, which means 7% of all Job rows described pods that could never have started. The k8s-2 release sizes the init container at 256 MiB plus the staged bytes.

Labels as limits. When we set the job container's limit to its label, 98 of 100 runs succeeded (Table 7). Both failures were process pools, where the label records the largest single process but the cgroup counts all of them together. One cell below the label, 25 of the 97 runs that started were OOM-killed.

Labels are sometimes a cell too generous. We also compared each corpus label with the job container's measured peak of anonymous plus shared memory, plus the same 5% headroom (Table 8). Depending on the staging, the label falls in the same cell for 52–68% of runs and within one cell for 91–100%. It is higher for 26–48% of runs and lower for at most 6%. Two things push labels up. The first is a quirk of `ru_maxrss`: on Linux, the value a parent reads for its child through `wait4` includes the parent's own high-water mark at the moment the child calls `exec`, because the kernel carries the old address space's peak into the new one. The measuring parent in our corpora held about 158 MiB, so even the smallest workloads report at least that much. The second is memory-mapped input files, which RSS counts but the kernel can reclaim; `numpy_linalg` and `image_batch` both map their inputs. Neither effect causes a run to fail at its label, but together they make about a third of labels one cell larger than strictly necessary. The oracle in Section 8 inherits this, so all regrets in this paper are measured against a slightly generous oracle.

Table 8: The corpus Job label compared with the job container's measured cgroup peak, in grid cells.

staging	n	same cell	within one	higher	lower
init-disk	23	65%	91%	30%	4%
init-tmpfs	31	68%	97%	26%	6%
boto3-disk	23	52%	96%	48%	0%
boto3-tmpfs	23	52%	100%	48%	0%

8 Cost model and metrics

We score a request by the dollars it takes to get the pod to finish successfully, counting failed attempts, retries and lateness, all simulated from the run's measurements. Each axis fails the way it fails on Kubernetes. If memory is too low, the container is OOM-killed partway through (at the point where the run reached 90% of its peak) and retried with double the memory, for up to eight attempts. If CPU is too low, the run is throttled: each second that wanted u cores takes $\max(1, u/\text{limit})$ seconds, replayed against the measured per-second series. If the GPU is too small, the run hits a CUDA out-of-memory error and is retried on the next machine up. A GPU task given no GPU fails at start and is retried on a T4, and a CPU task given a GPU runs normally but pays the GPU node's price.

A CPU pod is priced by its dominant share of an `m6i.xlarge` (4 vCPU, 16 GiB, $0.192 per hour), that is $p(m, c) = 0.192 \cdot \max(c/4,\ \text{GiB}(m)/16)$. A GPU pod pays the on-demand price of its machine. Every attempt also pays 0.02 hours for image pull and start-up. We normalize every run to one nominal hour of work, so that short measured runs stand in for longer production tasks.

On top of dollars, two penalties account for time. Each retry adds one oracle cost, which stands for the delay of running again, and each hour of lateness adds one oracle hourly price. These defaults match an orchestrator that retries automatically. A raw Job with `backoffLimit: 0` simply fails and waits for someone to resubmit it, so in Section 11.6 we also score everything with retry penalties of 5 and 20 oracle costs.

The *oracle* requests the label cell on each axis, and no GPU for a CPU task. It is the label rather than the cheapest possible request. Asking for one CPU cell below the busiest window only throttles that window, which can cost less than the cores it saves, so CPU regret can occasionally be slightly negative.

Metrics. Our headline metric is *regret*, the mean over runs of an approach's cost divided by the oracle's cost, minus one. Every run counts equally however much it costs, and this is also the quantity that every approach tunes on validation. *Dollar regret* is total cost over total oracle cost, minus one, so large jobs count for more. We also report the share of runs whose first attempt succeeds, and how often the memory request fits and is at most one cell above the label. For pairs of pods that come from the same run but differ in one detail of the pod spec, we check whether an approach's memory request moves in the same direction as the label (Section 11.3). We report this separately for pairs whose label moved and pairs whose label did not.

Uncertainty. Runs of the same task share code, and a run's Job and Flyte pods share a workload, so the rows are not independent. Every interval in this paper is therefore a 95% percentile interval from a task-clustered Poisson bootstrap [3, 4] with 2,000 replicates. Each task, including all of its runs and both pods of each run, gets a single Poisson(1) weight. Differences between approaches and between pod types are bootstrapped as paired differences on the same rows. We report variation between trainings separately, across three seeds (Section 11.7).

9 Approaches

Every approach produces a full request for every pod, and every setting an approach tunes is chosen on the validation split by mean regret. Appendix C lists the settings.

9.1 Static requests and rules

The simplest approach, **static**, gives every pod the same request: a training quantile of memory need and of CPU need, and no GPU. Every GPU task therefore fails at start and is retried on a T4.

Rules are what a platform team might write by hand. If the code mentions CUDA, the task gets a GPU sized from its largest past GPU peak times a margin. Memory is the largest past peak, converted to this pod's units and multiplied by a margin, or the static request if there is no history. CPU is the largest past peak in cores times a margin. On a first run it falls back to the largest pool size the code declares (`max_workers=`, `n_jobs=` and so on), or one core.

We added a stronger baseline, **rules-sized**, which also reads the pod spec. When a task has history, it takes the earlier run closest in input size and scales its peak by the ratio of input sizes. On a first run it estimates the workload's peak as $a + k \cdot$ input MiB, with a and k tuned on validation cold starts, and adds the staging's cost as read from the pod spec: 48 MiB if `job.py` imports boto3, and the staged bytes if the manifest says `medium: Memory`. The result is converted into the pod's need with the accounting of Section 6.2.

9.2 Gradient-boosted quantile trees

The trees use 209 features that a scheduler can compute. Twenty-seven describe the inputs and history, such as total bytes, bytes by format, visible rows, column counts, array shapes, the number of earlier runs, their peaks, and the size ratio to the closest earlier run. Another 173 describe the code, marking API tokens such as `groupby`, `TfidfVectorizer` or `ProcessPoolExecutor` and numeric literals such as pool sizes, layer widths and batch sizes. Four flag CUDA use, and five flag details of a Job's pod spec (`kind: Job`, `initContainers`, `medium: Memory`, `import boto3` and `download_file`).

For each axis we fit 19 quantile regressors [12] with scikit-learn's `HistGradientBoostingRegressor` (quantiles 0.05 to 0.95, 300 rounds, learning rate 0.06, 31 leaves) on the base-2 log of the need, plus a classifier that predicts whether the task needs a GPU. The predicted quantiles define a piecewise-linear CDF, and each cell's probability is the CDF mass between its edges. Two ablations, `gbt-dist-k8s` and `gbt-dist-flyte`, train on Job pods only or on Flyte pods only.

9.3 Zero-shot 27B

A 27B-parameter Qwen instruction model, served with vLLM [16], reads the prompt shown in Appendix B and answers in JSON at temperature zero. We parse its answer, read units as Kubernetes would (so `2 GiB`, `2GB` and `500m` all work), and round each value up onto the grid. An answer we cannot parse falls back to 4 Gi, one core and no GPU. The model reasons in plain text before answering even with thinking turned off, and our first release cut it off at 1,024 tokens, which left 444 of 560 answers unparseable. Here we score it two ways: with up to 8,192 tokens of free text, and with decoding constrained to the answer's JSON schema and 256 tokens. Both are scored on a family-stratified sample of 300 runs per split, with each run's Job and Flyte pods.

9.4 Decision model: three pointer heads on one forward pass

Instead of generating an answer, the decision model reads the prompt and scores every option on every axis in one forward pass (Figure 2). The options are written out in the prompt itself. For each axis, a small head

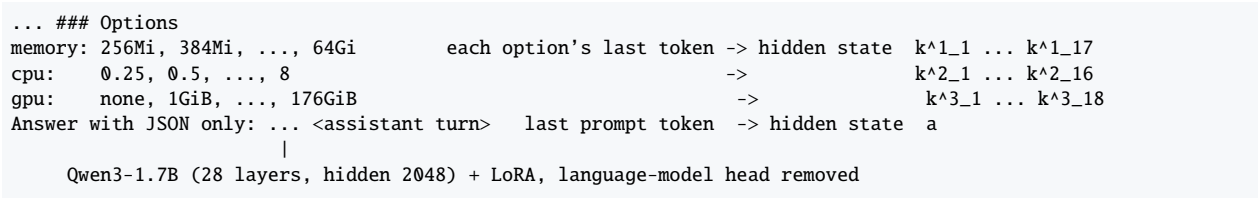


Figure 2: The pointer decision model. Every prompt ends with the same options block and answer instruction, so each option token sits at a fixed offset from the end.

compares the hidden state at each option's last token with the hidden state at the end of the prompt, so the model reads the candidates in its context rather than memorizing a fixed output layer:

$$s_{\text{axis}}(j) = \frac{\langle W_{q,\text{axis}}\,\text{LN}(a),\ W_{k,\text{axis}}\,\text{LN}(k_j)\rangle}{\sqrt{256}}, \qquad P(\text{axis} = j) = \text{softmax}_j\big(s_{\text{axis}}(j)/T_{\text{axis}}\big). \tag{1}$$

Here a is the last prompt token's hidden state, k_j is option j's, each axis has its own $W_q, W_k \in \mathbb{R}^{256\times 2048}$, and the temperature T_{axis} is fitted on validation. This is a pointer mechanism [23] applied to option tokens.

The torso is Qwen3-1.7B [17] without its language-model head, adapted with LoRA [10] (rank 16, α 32) on every projection of every layer. Prompts are cut from the left at 3,072 tokens, a length no evaluated prompt reaches. We train with cross-entropy on the three labels, weighting the CPU term by 0.3 and leaving out memory and CPU for GPU tasks. Training uses at most 5,000 rows per corpus and pod type, 27,936 rows in all, for one epoch with micro-batches of 8, two accumulation steps, and AdamW at 2×10^{-4} for the adapters and 10^{-3} for the heads, on one NVIDIA L4. Each axis's temperature minimizes validation negative log-likelihood.

Two ablations train on one pod type only: `jev-pointer-k8s` on Jobs and `jev-pointer-flyte` on Flyte tasks. Each has half the rows and trains for two epochs, so that all three variants take the same number of steps, and each is calibrated on its own pod type's validation rows.

9.5 Turning a distribution into a request

The trees, the decision models and the pools all produce a probability distribution on each axis, and we read each one out in two ways. The *argmax* read-out takes the most likely cell on each axis. The *decoded* read-out takes cost into account. It first decides whether to give a GPU, choosing none when $P(\text{none})$ is at least a threshold θ tuned on validation, and otherwise the GPU cell most likely to hold the peak. For memory it picks the cell with the lowest expected cost, $\arg\min_m \sum_j C[m, j]\, P(j)$, where $C[m, j]$ is the cost, retries included, of requesting cell m for a pod that needs cell j. For CPU it picks the cell most likely to fall within 25% of the busiest window. Because an OOM costs a retry, decoding leans toward asking for a little more memory, while argmax ignores that asymmetry.

9.6 Reinforcement learning against the cost model

Training to match the label misses two things the cost model cares about. The axes interact, since memory and CPU are free on a GPU node, while a GPU is the most expensive mistake on a CPU node. And asking for too little costs more than asking for too much. The RL approach (`rl-pointer`) starts from the decision model and directly maximizes the cost model's reward using GRPO [19], as shown in Figure 3. Because the policy is three categorical distributions, a group of 16 samples comes from a single forward pass and no text is generated. It trains on 6,000 prompts for one epoch with a KL coefficient of 0.05 and the reward capped at 20.

```
pi_ref = the supervised model's distributions, computed once before any update
for each batch of 8 prompts:
    one forward pass -> pi(.|x) on each axis
    sample K = 16 joint requests a^k = (m, c, g) from the three heads        # no text is generated
    r^k = - min( cost(x, a^k) / oracle_cost(x), 20 )                           # the leaderboard's simulator
    A^k = (r^k - mean_k r) / (std_k r + 1e-3)                                  # group-normalized
    L   = - mean_k[ A^k * sum_{axis in S(x)} log pi(a^k_axis | x) ] + 0.05 * sum_{axis in S(x)} KL(pi ||
     pi_ref)
    S(x) = {gpu} for a GPU task, else all three axes
```

Figure 3: GRPO without generation. The KL term is exact over the categorical distributions.

9.7 Opinion pools

The trees and the language models make different mistakes. The trees read numbers exactly but have nothing to go on for code they have never seen, while the language models read code well but treat numbers as tokens. A logarithmic opinion pool [7] combines their calibrated distributions on each axis:

$$p_{\text{pool}}(j) \propto p_{\text{trees}}(j)^{w}\, p_{\text{LM}}(j)^{1-w}. \tag{2}$$

A single weight w is shared across the three axes and tuned on validation together with θ, and nothing is retrained. A model trained on one pod type is pooled with trees trained on the same pod type and tuned on that pod type's validation rows.

9.8 A pre-registered novelty gate

In our first release, the pooled RL model read by argmax was the best approach on familiar code and one of the worst on new code, while the RL model on its own was the reverse. The gate sends each pod to one or the other according to how far it is from the training data:

$$s(x) = \tfrac{1}{10}\textstyle\sum_{i=1}^{10} \|z(x) - z(x_{(i)})\|_2, \qquad \text{request} = \begin{cases} \text{pool} \times \text{RL, argmax} & \text{if } s(x) \le \tau, \\ \text{RL, decoded} & \text{otherwise.} \end{cases} \tag{3}$$

Here z standardizes the trees' 209 features by their training mean and standard deviation, $x_{(i)}$ are the ten nearest training pods, and τ is the 95th percentile of s on validation. No labels are involved. Because validation contains familiar families only, the threshold sends exactly 5% of familiar pods to the RL model by construction. We chose the gate's members, signal and threshold after seeing the first release's results, so we fixed all three before training k8s-2 at new seeds, which makes k8s-2 a confirmation of a pre-registered choice.

An internal review found a problem with the first release's version. A feature that never varied in training was divided by 10^{-9}, so any pod with an unseen import landed effectively infinitely far away. We now leave such features in their raw units, and we keep the old scaling as `knn-raw` for comparison. We also compare two other signals: kNN on the 27 numeric features alone, and the Jensen–Shannon divergence between the trees' and the RL model's predictions.

9.9 A Flyte-to-Job adapter

A team whose history is all Flyte tasks has no Job pods to train on. The adapter shifts a Flyte-trained model's memory distribution using the accounting of Section 6.2, reading only what a scheduler can see: the manifest's volume medium, whether `job.py` imports boto3, and the input sizes. Each cell's representative need n_j (its geometric middle) becomes $(n_j/1.05 - R + 48[\text{boto3}] + B[\text{tmpfs}]) \times 1.05$. CPU and GPU come from the model's own reading of the Job's prompt.

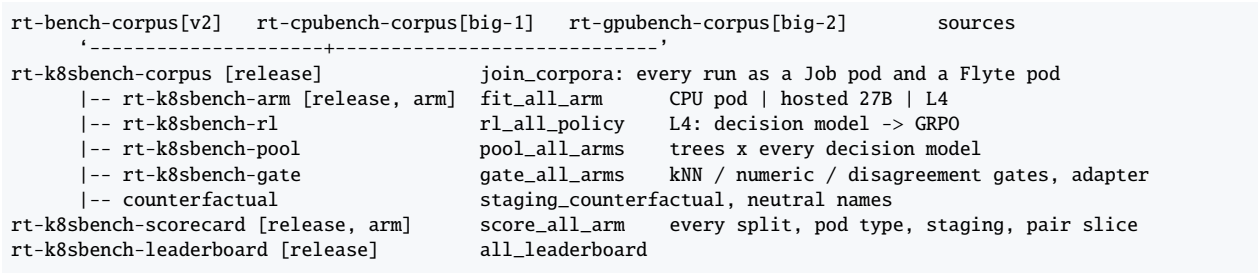


Figure 4: The benchmark as a Flyte factory. Re-running it rebuilds only what changed.

10 Experimental setup

The whole benchmark runs as one Flyte factory on an EKS cluster: a graph of cached tasks, each of which publishes a versioned artifact (Figure 4). The trees and rules fit on a CPU pod, each decision model and the RL model train on one NVIDIA L4, and the zero-shot model runs as a shared vLLM service in the same cluster. Each of these is itself a batch pod that someone had to size.

We trained k8s-2 three times on the same data (`rsk-k8s-2`, `-s2` and `-s3`). The seed controls data order, LoRA initialization and RL sampling. GPU kernels are not bit-for-bit deterministic, so even the same seed would not reproduce a model exactly. The zero-shot model is scored once, with seed 1. Unless we say otherwise, tables show seed 1 with its bootstrap interval, and Section 11.7 reports the spread across all three seeds. Appendix D lists every cluster run.

11 Results

We report Job pods first, because they are the plain Kubernetes case and the one with no orchestrator to absorb mistakes. Flyte pods follow, and so does the comparison between the two. All numbers are regret in percent with 95% task-clustered intervals, from seed 1 unless noted.

11.1 Leaderboard

Table 9 and Figure 5 give the main results. On new tasks from familiar families, pooling the trees with a model is clearly the best approach. Pooling them with the decision model reaches 6.6% on Job pods, and pooling them with the RL model and reading it out by argmax reaches 7.2%. Those two are statistically tied, and both are well ahead of the trees on their own (11.5%), the decision model on its own (12.6%), and the rules (42.4%). A static request costs almost twice the oracle.

New kinds of code tell a different story. On the unseen families, every approach does much worse, and the order changes. The RL model (39.3%) and our new `rules-sized` baseline (39.2%) are tied for best, with the zero-shot model close behind on its smaller sample (42.8%, with a wide interval). The pools, which led on familiar code, fall to 50–79%, and the trees fall to 69%. In other words, what works best on familiar code works poorly on new code, and a careful hand-written rule that reads input sizes and the pod spec is as good as anything we trained once the code is unfamiliar.

The pre-registered gate does what it was designed to do. It reaches 8.1% on familiar code, in the top group though not first, and 39.3% on new code, where it ties the best approach. No other trained approach comes as close to the top on both splits. On new code it does not beat the RL model it routes to, and it does not beat `rules-sized`; its value is that it gives up very little on familiar code to get there (Section 11.4).

Table 9: Regret (%) on Job pods, with 95% task-clustered intervals, for seed 1; regret on Flyte pods; dollar regret on Job pods; and the mean and range of Job regret over three trainings. Each split has 5,095 Job pods for test and 6,416 for unseen. Zero-shot is scored on a sample of about 300 runs per split, so it is not bolded.

	Job pods: regret (%)		Flyte pods		Job $ regret		Job, 3 seeds: mean (range)	
approach	test	unseen	test	unseen	test	unseen	test	unseen
static	89.3 [84.7, 93.7]	115.1 [111.2, 119.2]	91.5	112.8	183.0	258.1	89.3	115.1
rules	42.4 [37.9, 46.7]	49.8 [47.0, 52.5]	41.3	48.0	56.9	120.3	42.4	49.8
rules-sized	37.7 [33.8, 41.6]	**39.2** [37.3, 41.2]	37.3	41.1	55.9	117.9	37.7	39.2
zero-shot 27B	48.8 [33.1, 67.2]	42.8 [29.6, 57.2]	35.7	52.0	86.0	30.0	—	—
zero-shot 27B, JSON	48.8 [31.9, 67.7]	44.4 [31.2, 59.2]	21.8	51.5	100.8	81.9	—	—
trees	11.5 [10.2, 12.8]	69.1 [66.3, 71.8]	14.6	69.0	30.6	**85.6**	11.5	69.1
decision	12.6 [11.3, 14.0]	47.8 [45.6, 50.4]	11.3	43.0	39.6	119.0	13.7 (12.6–15.0)	49.5 (44.2–56.6)
decision, argmax	30.3 [28.0, 32.5]	69.1 [66.2, 72.1]	28.7	63.8	39.8	107.2	30.8 (30.3–31.7)	72.6 (68.1–80.6)
RL	14.6 [13.2, 16.1]	39.3 [37.3, 41.4]	14.3	**40.2**	50.6	127.9	16.2 (14.6–17.4)	37.7 (35.5–39.3)
RL, argmax	8.5 [7.1, 9.9]	46.5 [44.2, 48.8]	8.6	46.2	35.3	106.4	8.4 (8.2–8.6)	45.0 (41.3–47.2)
pool × decision	**6.6** [5.6, 7.6]	56.8 [54.5, 59.2]	**5.5**	51.4	27.6	95.0	6.8 (6.6–6.9)	55.7 (52.1–58.3)
pool × RL	10.5 [9.4, 11.8]	49.6 [47.5, 51.8]	11.1	50.3	34.7	100.8	10.7 (10.5–11.0)	53.9 (49.6–61.3)
pool × RL, argmax	7.2 [6.0, 8.4]	79.4 [75.7, 83.2]	7.1	78.1	**15.7**	111.6	8.6 (6.8–11.8)	89.6 (78.6–110.9)
gate (kNN, q 0.95)	8.1 [6.9, 9.4]	39.3 [37.3, 41.4]	7.6	**40.2**	19.5	127.9	9.6 (8.1–12.7)	37.7 (35.5–39.3)

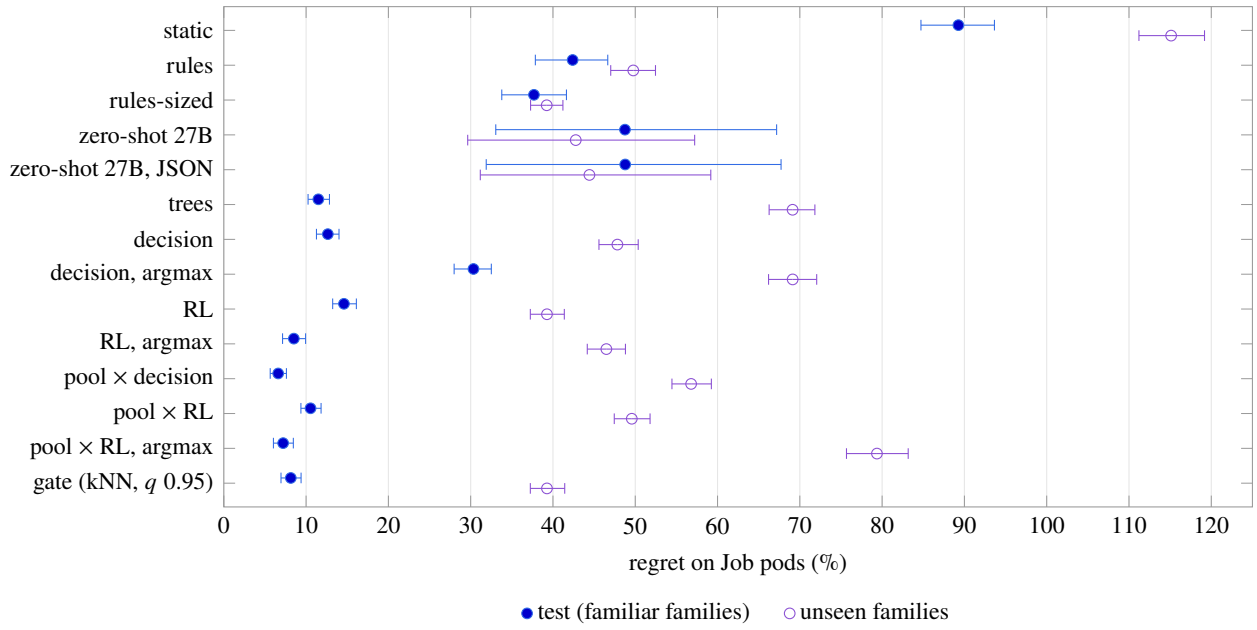


Figure 5: Regret on Job pods for seed 1, with 95% task-clustered intervals. Approaches that lead on familiar code (filled) are not the ones that lead on new code (open).

Table 10: What the training data includes. Regret (%) on Job and Flyte pods, and the share of Job pods whose first attempt succeeds, for seed 1.

		Job pods: regret (%)		Flyte pods: regret (%)		Job
approach	trained on	test	unseen	test	unseen	first OK
trees	both	11.5 [10.2, 12.8]	69.1	14.6 [13.3, 16.1]	69.0	97.6
	Jobs only	11.8 [10.5, 13.1]	68.6	26.2 [24.4, 28.0]	78.9	97.7
	Flyte only	18.3 [16.8, 19.9]	73.5	15.9 [14.5, 17.3]	93.5	94.1
decision	both	12.6 [11.3, 14.0]	47.8	11.3 [10.0, 12.6]	43.0	97.2
	Jobs only	13.2 [11.8, 14.7]	56.1	20.5 [18.8, 22.3]	53.6	97.0
	Flyte only	37.3 [35.1, 39.6]	59.0	12.0 [10.6, 13.5]	48.5	87.2
decision, argmax	both	30.3 [28.0, 32.5]	69.1	28.7 [26.5, 31.0]	63.8	87.3
	Jobs only	28.8 [26.7, 31.1]	74.4	41.4 [38.8, 44.0]	79.3	87.0
	Flyte only	56.2 [53.3, 59.3]	84.4	27.5 [25.4, 29.6]	66.6	76.9
pool × decision	both	6.6 [5.6, 7.6]	56.8	5.5 [4.6, 6.6]	51.4	98.3
	Jobs only	7.7 [6.6, 8.8]	57.9	17.1 [15.6, 18.6]	58.7	98.0
	Flyte only	24.8 [22.9, 26.7]	51.6	5.4 [4.4, 6.5]	49.6	91.3

11.2 Does a model need to see Jobs?

Table 10 trains the trees, the decision model and the pool three ways: on both pod types, on Jobs alone, and on Flyte tasks alone. Two results stand out. First, the Flyte data does not help with Jobs. A decision model trained only on Jobs scores 13.2% on Job test pods, against 12.6% for one trained on both, a difference of 0.6 points with an interval that includes zero ($[-1.8, +0.7]$). The trees and the pool behave the same way. Second, the reverse transfer does not work. A model trained only on Flyte tasks scores 37.3% on Job pods, three times worse than one that has seen Jobs, and the pool built on it reaches 24.8% where the both-pod-types pool reaches 6.6%. A model trained only on Jobs also does worse on Flyte pods (20.5% against 11.3%), but by much less, because a Flyte task is closer to a Job with a small fixed runtime added than the other way around.

A model trained on both pod types treats them almost the same. The paired difference in regret between a run's Job pod and its Flyte pod is at most 1.4 points on familiar code for the language models and pools (Table 18 in Appendix E), and for the RL model it is indistinguishable from zero. The trees do 3.1 points better on Jobs than on Flyte tasks. On new code the decision model and its pool do about 5 points worse on Jobs than on Flyte tasks, while RL and the gate show no gap.

The adapter. A team whose history is all Flyte tasks has no Job pods to learn from. The adapter of Section 9 shifts a Flyte-trained model's memory distribution through the staging accounting, and with the corrected 70 MiB runtime it now helps on every seed. It cuts the Flyte-only model's Job regret from 37.3% to 28.6% on seed 1, from 33.8% to 22.1% on seed 2 and from 37.5% to 28.6% on seed 3. With the 180 MiB constant of our first release it made Job regret worse, which in hindsight was a symptom of that wrong constant. The adapter still falls well short of a model that has seen Jobs (12–14%), though, so platform data matters even when the accounting is right.

11.3 Does a model read the manifest?

A model could do well on Jobs for the wrong reason, for example by learning only that a Job's numbers run lower than a Flyte task's because a Job does not carry the runtime. We test this two ways.

Table 11: The staging counterfactual for seed 1. "Moved, right" is the share of pairs whose label moved when the volume switched from disk to tmpfs where the approach's memory request moved the same way. "Unmoved, kept" is the share of pairs whose label stayed put where the request stayed put too. "Fetcher moved" is the first measure for a switch from an init container to boto3.

	test			unseen		
approach	moved, right	unmoved, kept	fetcher moved	moved, right	unmoved, kept	fetcher moved
static	0.0	100.0	0.0	0.0	100.0	0.0
rules	0.0	100.0	0.0	0.0	100.0	0.0
trees	74.7	76.6	30.8	78.1	20.6	24.0
decision	73.6	76.6	44.6	72.6	30.6	37.3
decision, Job-only	77.3	79.0	40.0	63.6	30.0	33.8
decision, Flyte-only	21.3	87.5	15.8	31.9	65.8	30.2
RL	72.2	66.8	35.8	71.6	42.6	29.8
pool × decision	77.3	77.4	39.6	77.4	20.3	32.4
pool × RL, argmax	77.6	82.0	46.7	72.7	31.9	32.0

Run and Job pairs. Of the 3,950 test pairs of a run's Flyte pod and its Job pod on CPU nodes, the memory label differs in only 29.9%. In 638 pairs the Job needs more, because it stages its inputs on tmpfs, and in 543 it needs less, because it has no runtime. With the old 180 MiB constant, 69% of pairs differed and almost all of those moves were downward, so this metric mostly measured whether a model knew about the runtime. A model that ignores the pod type keeps the same request for both pods; it scores 100% on the pairs whose label stays the same and 0% on the pairs whose label moves. Models trained on both pod types move the right way on 56–67% of the pairs that move, and on 75–83% of upward moves (Table 19 in Appendix E). The rules move the right way on 25%.

A controlled counterfactual. The pairs above still mix the effect of the manifest with the effect of the runtime. To isolate the manifest, we took 1,189 first runs from the test and unseen splits and rendered each one as a Job under all four stagings, holding the task, code and inputs fixed (4,756 pods in all). Only the manifest and the way the job fetches its inputs change. Because these are first runs, there is no history to give the staging away, so the manifest is the only evidence. Switching the volume from disk to tmpfs moves the label in 48.6% of test pairs and 74.2% of unseen pairs, and switching the fetcher from an init container to boto3 moves it in about a fifth.

Table 11 shows that the models trained on Jobs do read the manifest. When the label moves, the decision model, the trees, RL and the pools all move their request the right way on 72–78% of familiar-code pairs and 64–78% of unseen pairs. The Flyte-only model, which has never seen a manifest, does so on only 21%. Static requests and rules never move, because on a first run they have nothing to go on but fixed defaults. The models are worse at the opposite case on new code. When the label does not move on unseen pairs, which mostly means the staged inputs were small, they still change their request on 57–80% of them, overreacting to `medium: Memory`. They also pick up the fetcher less reliably (31–47% on test), which makes sense since boto3's 48 MiB only occasionally crosses a cell.

The task's name. Generated tasks are named after their family (for example `pandas-join`), and the name appears in both the code and the manifest. Real tasks have names too, but ours give away the family. We therefore re-scored the same first runs with each name replaced by a hash, without retraining anything (Table 20 in Appendix E). On familiar code, regret changes by less than a point for every trained approach except the Flyte-only model (+1.8). On new code it rises by 0.5–3.6 points. The models lean on the name a little when the code is unfamiliar, but the name is not what they rely on.

Table 12: Gate signals for seed 1. AUROC measures how well each signal ranks unseen pods above validation pods. The middle columns give the share of pods routed to the RL model, and the last two the gate's regret on Job pods. All gates use the 0.95 quantile except the disagreement gate, shown at 0.5.

		sent to RL (%)		Job regret (%)	
signal	AUROC	test	unseen	test	unseen
kNN, all features	0.999	5.3	100.0	8.1 [7.0, 9.4]	39.3 [37.3, 41.3]
kNN, k8s-1 scaling	1.000	5.3	100.0	8.1 [7.0, 9.4]	39.3 [37.1, 41.4]
kNN, numeric only	0.673	4.5	30.5	7.9 [6.7, 9.2]	61.9 [59.0, 64.7]
disagreement (JS), q 0.5	0.729	51.3	80.5	11.5 [10.2, 13.0]	42.9 [40.8, 45.0]

11.4 The pre-registered gate

We fixed the gate before training k8s-2, so its results are a confirmation rather than a fit. Table 12 compares its signal with the alternatives. Correcting the kNN scaling, which an internal review suspected was behind the near-perfect detection in our first release, changed nothing. The corrected distance still separates unseen families from validation with an AUROC of 0.999, sends every unseen pod to RL, and gives exactly the same requests. The held-out families really are far from the training code. Six of the seven import a library that no training family uses, and the remaining code and input features put them several times further away than any familiar pod. The detector is therefore easy on this benchmark, and we cannot measure it on new code that uses only familiar libraries. With code tokens removed, the AUROC falls to 0.67 and unseen regret rises to 62%.

Over three trainings, the gate scores 9.6% on familiar code (8.1–12.7%) and 37.7% on new code (35.5–39.3%). Seed 3's familiar-code number is higher because its pooled RL model, the gate's member for familiar code, was worse at 11.8%. On new code the gate always equals the RL model, so its quality there is RL's. In paired terms on seed 1, the gate costs 1.5 points more than the best pool on familiar code and saves 17.5 points on new code.

11.5 First runs and runs with history

Cold starts are where the approaches differ most (Table 13). With history, almost everything trained does well on familiar code, from 1.6% for RL read by argmax to 10.1% for RL decoded, and even `rules-sized` reaches 11.6%. On first runs the rules lose most of their advantage (66–73%), while the pooled decision model holds at 10.8%. On new code, history does most of the work. `rules-sized` with history reaches 13.8%, which is the best of any approach, because scaling the nearest earlier run by input size is a strong prior when a task's runs differ only in size. Without history, no approach does well on new code. The best are the zero-shot model (57.7%, on a small sample) and RL (61.4%).

11.6 Dollars and the price of a retry

Mean regret counts every run equally, while dollar regret is dominated by large jobs. On familiar code the two agree: the pooled RL model read by argmax costs 15.7% more than the oracle in dollars, and the gate 19.5%. On new code, dollars rank the trees first (85.6%), because they size the largest jobs better, even though their mean regret is among the worst. The gate and RL cost $1,177 per 1,000 unseen Job runs, against $959 for the trees, $1,126 for `rules-sized` and $517 for the oracle.

A raw Job with `backoffLimit: 0` does not retry; it fails, and someone resubmits it. That makes an OOM much more expensive than our default penalty of one oracle cost per retry. Table 14 re-scores everything with penalties of 0, 5 and 20 oracle costs. The pooled decision model stays best on familiar code at every

Table 13: Job regret (%) on first runs and on runs with history, for seed 1.

	test		unseen	
approach	first run	with history	first run	with history
static	98.4	82.5	131.7	100.8
rules	66.1	24.6	73.3	29.4
rules-sized	72.5	11.6	68.6	**13.8**
zero-shot 27B	84.2	18.4	57.7	32.2
zero-shot 27B, JSON	75.8	25.6	79.8	19.5
trees	16.1	8.1	110.5	33.3
decision	21.7	5.8	77.3	22.3
decision, argmax	48.1	17.0	108.5	35.0
RL	20.6	10.1	**61.4**	20.1
RL, argmax	17.7	**1.6**	76.0	21.0
pool × decision	**10.8**	3.5	99.4	19.8
pool × RL	13.7	8.2	85.8	18.2
pool × RL, argmax	13.5	2.5	140.7	26.2
gate (kNN, q 0.95)	14.5	3.4	**61.4**	20.1

Table 14: Job regret (%) when each OOM retry costs 0, 1 (the default), 5 or 20 oracle costs. Higher penalties stand for a Job that fails outright and has to be resubmitted.

	0		1 (default)		5		20	
approach	test	unseen	test	unseen	test	unseen	test	unseen
static	47.8	44.3	89.3	115.1	255.2	398.2	877.6	1459.9
rules	28.8	24.7	42.4	49.8	96.9	149.9	301.2	525.6
rules-sized	25.9	20.9	37.7	**39.2**	84.9	**112.5**	261.8	**387.2**
zero-shot 27B	31.1	18.8	48.8	42.8	119.5	138.8	384.9	498.8
zero-shot 27B, JSON	26.5	24.4	48.8	44.4	138.0	124.4	472.6	424.4
trees	6.5	50.8	11.5	69.1	31.4	142.5	105.8	417.7
decision	7.3	20.2	12.6	47.8	33.8	158.6	113.3	573.8
decision, argmax	17.8	35.2	30.3	69.1	80.6	204.8	269.0	713.5
RL	8.1	**15.2**	14.6	39.3	40.4	135.7	137.3	497.1
RL, argmax	**1.9**	19.1	8.5	46.5	35.0	156.1	134.6	567.1
pool × decision	2.3	37.0	**6.6**	56.8	**24.0**	135.9	**89.4**	432.6
pool × RL	5.6	31.0	10.5	49.6	30.4	124.0	104.9	403.2
pool × RL, argmax	2.3	54.3	7.2	79.4	26.9	179.6	100.8	555.6
gate (kNN, q 0.95)	2.9	**15.2**	8.1	39.3	29.1	135.7	107.7	497.1

penalty from 1 to 20, at 89.4% even when a retry costs twenty times the oracle. (With no retry penalty at all, RL read by argmax is best, at 1.9%.) On new code, a larger penalty favours the approaches that ask for more memory. At a penalty of 5 or 20, `rules-sized` becomes clearly the best approach on unseen code (112.5% and 387.2%), ahead of RL and the gate (135.7% and 497.1%). For teams that run raw Jobs without retries, this is the most practical result in the paper. On code nobody has seen before, a rule that scales by input size and reads the manifest is safer than any model we trained.

Table 15: Job regret (%) for each of the three trainings. The deterministic approaches give identical results on every seed.

	Job test, by seed			Job unseen, by seed		
approach	1	2	3	1	2	3
static	89.3	89.3	89.3	115.1	115.1	115.1
rules	42.4	42.4	42.4	49.8	49.8	49.8
rules-sized	37.7	37.7	37.7	39.2	39.2	39.2
trees	11.5	11.5	11.5	69.1	69.1	69.1
decision	12.6	13.4	15.0	47.8	44.2	56.6
decision, argmax	30.3	30.3	31.7	69.1	68.1	80.6
RL	14.6	16.7	17.4	39.3	38.2	35.5
RL, argmax	8.5	8.6	8.2	46.5	47.2	41.3
pool × decision	6.6	6.8	6.9	56.8	52.1	58.3
pool × RL	10.5	10.5	11.0	49.6	50.9	61.3
pool × RL, argmax	7.2	6.8	11.8	79.4	78.6	110.9
gate (kNN, q 0.95)	8.1	8.1	12.7	39.3	38.2	35.5
decision, Job-only	13.2	13.8	12.2	56.1	53.3	68.5
decision, Flyte-only	37.3	33.8	37.5	59.0	67.1	76.1
trees, Job-only	11.8	11.8	11.8	68.6	68.6	68.6
trees, Flyte-only	18.3	18.3	18.3	73.5	73.5	73.5

11.7 Three trainings

The deterministic approaches (static, both rules and the trees) give identical results on every seed. The trained models vary, much more on new code than on familiar code (Table 15). On familiar code, the pooled decision model ranges from 6.6% to 6.9% and RL read by argmax from 8.2% to 8.6%, so the top of the familiar-code leaderboard is stable. The pooled RL model read by argmax is less so, ranging from 6.8% to 11.8%. On new code, the decision model ranges from 44% to 57% and the pooled RL model from 79% to 111%, while RL itself varies least (35.5–39.3%). Rankings on new code within about 10 points of each other should therefore not be read as stable, with the exception of RL and the gate, which are consistently near the top.

11.8 Zero-shot

With room to finish its reasoning, the 27B model answers almost every prompt, and the results are no longer dominated by fallbacks. It scores 48.8% on familiar-code Job pods and 42.8% on unseen ones, against 42.4% and 49.8% for the rules. Constraining it to the JSON schema makes little difference on Jobs (48.8% and 44.4%) but helps on Flyte tasks (21.8% on familiar code). Its intervals are wide, about 30 points, because it is scored on a sample, so the honest summary is that a large model with no training is roughly as good as hand-written rules and much worse than a small model trained on the benchmark. It is, however, the only approach that does not get worse on new code.

12 Discussion

Benchmarking Kubernetes pods from one set of measurements. Because the user code is the same whichever way a pod is submitted, one measurement of the workload can label both a raw Job and a Flyte task, as long as the accounting for the pod's edges is right. Getting it right took real pods. Sibling processes in one pod confirmed the arithmetic, but only per-container cgroups showed where tmpfs pages are charged, that the first release's init containers could not hold their inputs, and that the Flyte runtime was less than half

the size we had assumed. Anyone deriving Kubernetes labels this way should check them against the cgroup, not against another RSS measurement.

What a model learns from Jobs. A model trained on Jobs reads the manifest. It changes its memory request when the volume medium changes, and it does so on first runs where nothing else gives the staging away. Flyte tasks add nothing to that model's accuracy on Jobs, and a Flyte-only model does not transfer well even after an explicit adapter. For a team that submits pods both ways, the practical advice is to train on the pods it actually runs. Flyte history is still useful for sizing Flyte tasks, and the corrected adapter recovers part of the gap when Job history is missing.

Familiar and unfamiliar code. The gap between the two splits is the main open problem. On familiar code the best approaches are within 7–8% of the oracle. On new kinds of code, the best trained model is 39% above it, and a careful rule matches it. Pooling the trees with a language model helps on familiar code and hurts on new code, because the pool's weight is tuned on familiar families. The gate recovers the better of the two on each split, but on this benchmark its detector has an easy job, since new families usually bring new libraries. A benchmark that separates unfamiliar code from unfamiliar libraries would test it properly.

Cost, not accuracy. Accuracy on the label and cost often disagree. Cost-aware decoding over-asks on purpose, the Flyte-to-Job adapter was judged a failure until its runtime constant was corrected, and the ranking on new code flips when retries become expensive. A benchmark for this decision has to score money, and it has to say what a failure costs.

13 Limitations

The workloads are synthetic. They use real libraries and real code paths, but the tasks are generated, the data is random, and runs take seconds to minutes. Production tasks are longer and messier. The seven unseen families were chosen by hand, and six of them introduce a library that no training family uses, so the benchmark does not test new code built only from familiar libraries. The people who wrote the generators also wrote the tree features and the rules, and the task names give away the family, although removing them changes regret by less than four points.

The labels carry three known biases. The `ru_maxrss` floor and memory-mapped inputs make about a third of labels one cell larger than strictly needed, so the oracle is slightly generous. Process pools are labelled by their largest process rather than their total, which is why two of 100 real pods failed at their label. And RightSizeMem has no per-second CPU series, so its throttling is modelled from mean cores. The Job labels are derived rather than measured for every run. We validated them on 100 runs in real pods and 176 in a single pod, but we did not re-measure GPU runs as Jobs, and the four stagings leave out sidecars, CSI drivers and other init work.

The cost model uses list prices, with no spot discounts or bin-packing, and prices a retry at one oracle cost by default; Section 11.6 shows how much that choice matters. We trained three times, which shows that rankings on new code within about ten points are not stable, but three seeds are not enough to put tight intervals on that variation. The zero-shot model was scored on a sample of about 300 runs per split, and only once.

14 Conclusion

RightSize Bench scores the request a scheduler makes before a Kubernetes pod runs, in dollars, across 33 families of task code submitted both as raw Jobs and through Flyte. Checking its labels in real pods fixed two errors and measured the cost of Flyte's runtime at about 70 MiB. A small model trained on Jobs reads the manifest and, pooled with gradient-boosted trees, comes within 7% of the oracle on familiar code. On unfamiliar code, the best trained model and a careful rule are both about 39% above the oracle, and the rule is safer when failures are expensive. Closing that gap, and testing a novelty detector on code that looks familiar but is not, is the work that remains.

References

[1] Eli Cortez, Anand Bonde, Alexandre Muzio, Mark Russinovich, Marcus Fontoura, and Ricardo Bianchini. Resource central: Understanding and predicting workloads for improved resource management in large cloud platforms. In *Proceedings of the 26th Symposium on Operating Systems Principles (SOSP)*, 2017. doi: 10.1145/3132747.3132772.

[2] Christina Delimitrou and Christos Kozyrakis. Quasar: Resource-efficient and QoS-aware cluster management. In *Proceedings of the 19th International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, 2014. doi: 10.1145/2541940.2541941.

[3] Bradley Efron and Robert J. Tibshirani. *An Introduction to the Bootstrap*. Chapman & Hall, 1993.

[4] Christopher A. Field and Alan H. Welsh. Bootstrapping clustered data. *Journal of the Royal Statistical Society: Series B (Statistical Methodology)*, 69(3):369–390, 2007. doi: 10.1111/j.1467-9868.2007.00593.x.

[5] Flyte Authors. Flyte. `https://flyte.org`, 2025.

[6] Jerome H. Friedman. Greedy function approximation: A gradient boosting machine. *The Annals of Statistics*, 29(5):1189–1232, 2001. doi: 10.1214/aos/1013203451.

[7] Christian Genest and James V. Zidek. Combining probability distributions: A critique and an annotated bibliography. *Statistical Science*, 1(1):114–135, 1986. doi: 10.1214/ss/1177013825.

[8] Chuan Guo, Geoff Pleiss, Yu Sun, and Kilian Q. Weinberger. On calibration of modern neural networks. In *Proceedings of the 34th International Conference on Machine Learning (ICML)*, 2017.

[9] Tejun Heo. Control group v2. Linux kernel documentation, `https://docs.kernel.org/admin-guide/cgroup-v2.html`, 2024.

[10] Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. LoRA: Low-rank adaptation of large language models. In *International Conference on Learning Representations*, 2022.

[11] Guolin Ke, Qi Meng, Thomas Finley, Taifeng Wang, Wei Chen, Weidong Ma, Qiwei Ye, and Tie-Yan Liu. LightGBM: A highly efficient gradient boosting decision tree. In *Advances in Neural Information Processing Systems*, volume 30, 2017.

[12] Roger Koenker and Gilbert Bassett. Regression quantiles. *Econometrica*, 46(1):33–50, 1978. doi: 10.2307/1913643.

[13] Kubernetes Authors. Resource management for pods and containers. https://kubernetes.io/docs/concepts/configuration/manage-resources-containers/, 2024.

[14] Kubernetes Authors. Volumes: emptyDir. https://kubernetes.io/docs/concepts/storage/volumes/#emptydir, 2024.

[15] Kubernetes Authors. Vertical pod autoscaler. https://github.com/kubernetes/autoscaler/tree/master/vertical-pod-autoscaler, 2024.

[16] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with PagedAttention. In *Proceedings of the 29th Symposium on Operating Systems Principles (SOSP)*, 2023. doi: 10.1145/3600006.3613165.

[17] Qwen Team. Qwen3 technical report, 2025.

[18] Krzysztof Rzadca, Pawel Findeisen, Jacek Swiderski, Przemyslaw Zych, Przemyslaw Broniek, Jarek Kusmierek, Pawel Nowak, Beata Strack, Piotr Witusowski, Steven Hand, and John Wilkes. Autopilot: Workload autoscaling at Google. In *Proceedings of the Fifteenth European Conference on Computer Systems (EuroSys)*, 2020. doi: 10.1145/3342195.3387524.

[19] Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, Y. K. Li, Y. Wu, and Daya Guo. DeepSeekMath: Pushing the limits of mathematical reasoning in open language models, 2024.

[20] Yiyou Sun, Yifei Ming, Xiaojin Zhu, and Yixuan Li. Out-of-distribution detection with deep nearest neighbors. In *Proceedings of the 39th International Conference on Machine Learning (ICML)*, 2022.

[21] Muhammad Tirmazi, Adam Barker, Nan Deng, Md E. Haque, Zhijing Gene Qin, Steven Hand, Mor Harchol-Balter, and John Wilkes. Borg: The next generation. In *Proceedings of the Fifteenth European Conference on Computer Systems (EuroSys)*, 2020. doi: 10.1145/3342195.3387517.

[22] Abhishek Verma, Luis Pedrosa, Madhukar Korupolu, David Oppenheimer, Eric Tune, and John Wilkes. Large-scale cluster management at Google with Borg. In *Proceedings of the Tenth European Conference on Computer Systems (EuroSys)*, 2015. doi: 10.1145/2741948.2741964.

[23] Oriol Vinyals, Meire Fortunato, and Navdeep Jaitly. Pointer networks. In *Advances in Neural Information Processing Systems*, volume 28, 2015.

A Task families

Table 16 lists every family with its resource driver and one sample input, rendered as an approach sees it (seed 7). "Unseen" families are held out of training and validation. "Sharded" families spread over every visible GPU. Lines is the length of the sample's function body.

Table 16: The 33 task families.

corpus	family	driver	sample input	lines
RightSizeMem	csv_chunked	code	readings: csv, header: device,x0,x1,x2,x3,x4,x5,x6,x7,tag0	13
	jsonl_records	both	logs: jsonl, first record keys: ['user', 'score0', 'scor...	17
	numpy_linalg	both	matrix: npy, shape=(326345, 128), dtype=float64	15
	pandas_groupby	both	events: parquet, 1,790,583 rows, schema=[customer_id: i...	14
	pandas_join	both	orders: parquet, 632,344 rows, schema=[order_id: int64,...	19
	polars_pipeline	code	events: parquet, 2,236,012 rows, schema=[user_id: int64...	15
	python_grouping	both	rows: csv, header: key,value,note0,note1,note2	18
	sklearn_fit	both	train: parquet, 326,345 rows, schema=[x0: double, x1: do...	19
	tfidf_text	both	docs: parquet, 52,765 rows, schema=[doc_id: int64, text...	16
	torch_train	both	train: parquet, 1,710,807 rows, schema=[x0: double, x1:...	39
	duckdb_sql (unseen)	code	events: parquet, 2,403,570 rows, schema=[account_id: in...	15
	image_batch (unseen)	both	images: npy, shape=(52998, 128, 128, 3), dtype=uint8	15
RightSizeCPU	io_bound	io	events: jsonl, first record keys: ['user', 'event', 'pay...	27
	mixed_phases	phases	matrix: npy, shape=(101438, 256), dtype=float64	28
	numpy_blas	threads	features: npy, shape=(135042, 512), dtype=float32	25
	pandas_serial	serial	orders: parquet, 893,225 rows, schema=[customer_id: int...	20
	polars_groupby	threads	events: parquet, 5,464,904 rows, schema=[key: int64, reg...	22
	process_pool	processes	docs: csv, header: id,text	31
	python_threadpool	gil	blob: npy, shape=(1773, 65536), dtype=uint8	29
	sklearn_ensemble	threads	train: parquet, 177,376 rows, schema=[f0: double, f1: do...	22
	torch_cpu	threads	train: parquet, 177,376 rows, schema=[f0: double, f1: do...	32
	duckdb_sql (unseen)	threads	ledger: parquet, 5,464,904 rows, schema=[account_id: in...	22
	pyarrow_csv (unseen)	threads	readings: csv, header: sensor,r0,r1,r2,r3	24
RightSizeGPU	blocked_gemm	blocks	vectors: npy, shape=(48071, 256), dtype=float32	28
	cnn_train	activations	images: npy, shape=(4409, 224, 224, 3), dtype=uint8	35
	encoder_infer	data	tokens: npy, shape=(44344, 512), dtype=int32	35
	gpt_train	activations	tokens: npy, shape=(17737, 1025), dtype=int32	55
	lm_finetune (sharded)	params	tokens: npy, shape=(1181, 1025), dtype=int32	63
	loader_bound	input_bound	images: npy, shape=(7681, 64, 64, 3), dtype=uint8	43
	mlp_tabular	params	train: parquet, 480,714 rows, schema=[f0: float, f1: flo...	36
	knn_search (unseen)	blocks	index: npy, shape=(240357, 128), dtype=float32	27
	llm_generate (unseen) (sharded)	kv_cache	prompts: npy, shape=(3313, 2048), dtype=int32	59
	recsys_train (unseen) (sharded)	params	clicks: parquet, 1,786,450 rows, schema=[user: int64, it...	53

B A complete prompt

The prompt every language-model approach reads, here for the init-tmpfs Job of a k8s-2 test task. The init container's memory limit is 281 Mi: 256 Mi plus the 25 MiB it stages onto the memory-backed volume. A Flyte pod's prompt shows the task module in place of `job.py` and the manifest, and reports history peaks with the runtime included.

System

```
You size batch jobs before they run: Flyte tasks, or Kubernetes Jobs given as a Python module and its manifest.
     Given the code, the metadata of this run's inputs, and what earlier runs of the same job used, choose the
     cheapest request that runs it: memory, CPU cores, and a GPU or none. Memory is a hard limit: below the
    peak the container is OOM-killed and retried with double. A Flyte task's memory includes the Flyte runtime
    ; a Kubernetes container's includes whatever the job loads to fetch its inputs, and files written to a
    memory-backed emptyDir (medium: Memory) are charged to it. CPU below the busiest 15 seconds' use throttles
     the task and it finishes late. Thread pools the code does not size get one thread. gpu is the peak GPU
    memory to provision, or none for a task that does not use CUDA; it lands on the cheapest machine that
    holds it -- T4 (15 GiB, $0.53/h), L4 (22 GiB, $0.98/h), A10G (22 GiB, $1.21/h), L40S (44 GiB, $2.24/h), 4
    xL4 (88 GiB, $4.60/h), 8xT4 (120 GiB, $7.82/h), 4xL40S (176 GiB, $10.49/h) -- and a GPU pod comes with its
     host memory and cores. A multi-GPU machine only helps code that spreads its model over every visible GPU.
     A GPU task given no GPU fails at start; a GPU given to a CPU task is paid for the whole run.
```

User

````
### Job code (job.py)
```python
import pandas as pd
````
````

```
import json
import os
import sys


def join_orders_catalog(orders_path: str, catalog_path: str) -> dict:
    orders = pd.read_parquet(orders_path)
    catalog = pd.read_parquet(catalog_path)
    catalog = catalog.drop_duplicates("sku", keep="last")
    joined = orders.merge(catalog, on="sku", how="inner")
    joined["revenue"] = joined["qty"] * joined["price"]
    by_brand = joined.groupby("brand", observed=True)["revenue"].sum()
    return {"rows": int(len(joined)), "brands": int(len(by_brand))}


if __name__ == "__main__":
    result = join_orders_catalog(sys.argv[1], sys.argv[2])
    os.makedirs("/data/out", exist_ok=True)
    with open("/data/out/result.json", "w") as f:
        json.dump(result, f)
```

Job manifest
```yaml
apiVersion: batch/v1
kind: Job
metadata:
  generateName: pandas-join-
  labels: {app.kubernetes.io/name: pandas-join}
spec:
  backoffLimit: 0
  template:
    spec:
      restartPolicy: Never
      initContainers:
        - name: stage-inputs
          image: amazon/aws-cli:2.17.0
          command: [sh, -c, "aws s3 cp $INPUT_ORDERS /data/orders.parquet && aws s3 cp $INPUT_CATALOG /data/
     catalog.parquet"]
          env: [{name: INPUT_ORDERS, value: "s3://batch-data/pandas-join/56ef2668/run-1/orders.parquet"}, {
     name: INPUT_CATALOG, value: "s3://batch-data/pandas-join/56ef2668/run-1/catalog.parquet"}]
          resources: {requests: {cpu: 500m, memory: 256Mi}, limits: {memory: 281Mi}}
          volumeMounts: [{name: data, mountPath: /data}]
      containers:
        - name: main
          image: registry.internal/batch/pandas-join:84b6a081
          command: [sh, -c, "python /app/job.py /data/orders.parquet /data/catalog.parquet && aws s3 cp /data/
     out/result.json $OUTPUT_URI"]
          env:
            - {name: OMP_NUM_THREADS, value: "1"}
            - {name: OPENBLAS_NUM_THREADS, value: "1"}
            - {name: MKL_NUM_THREADS, value: "1"}
            - {name: OUTPUT_URI, value: "s3://batch-data/pandas-join/56ef2668/run-1/out/"}
          resources: {}  # to be decided
          volumeMounts: [{name: data, mountPath: /data}]
      volumes:
        - name: data
          emptyDir: {medium: Memory}
```

This run's inputs
```
orders: parquet, 15.48 MiB, 632,510 rows, schema=[order_id: int64, sku: int64, qty: int32, price: double, ts:
     int64]
catalog: parquet, 9.43 MiB, 80,000 rows, schema=[sku: int64, attr0: double, attr1: double, attr2: double,
     attr3: double, attr4: double, attr5: double, attr6: double, attr7: double, attr8: double, attr9: double,
     attr10: double, attr11: double, brand: string (dictionary)]
```

Earlier runs of this job (container memory peaks, from the cluster's metrics)

```
- inputs orders: parquet, 49.05 MiB, 2,013,227 rows; catalog: parquet, 9.43 MiB, 80,000 rows -> peak memory
     668 MiB, mean 0.99 cores, 1 s

### Options
memory: 256Mi, 384Mi, 512Mi, 768Mi, 1Gi, 1.5Gi, 2Gi, 3Gi, 4Gi, 6Gi, 8Gi, 12Gi, 16Gi, 24Gi, 32Gi, 48Gi, 64Gi
cpu: 0.25, 0.5, 0.75, 1, 1.25, 1.5, 1.75, 2, 2.5, 3, 3.5, 4, 5, 6, 7, 8
gpu: none, 1GiB, 1.5GiB, 2GiB, 3GiB, 4GiB, 6GiB, 8GiB, 10GiB, 12GiB, 15GiB, 22GiB, 32GiB, 44GiB, 64GiB, 88GiB,
      120GiB, 176GiB

Answer with JSON only: {"memory": "<one memory option>", "cpu": "<one cpu option>", "gpu": "<one gpu option>"}
```

C Hyperparameters and tuned settings

Table 17: Settings. Tuned values are seed 1's, and they were identical across the three seeds except where noted.

approach	setting	value
static	quantiles (tuned)	memory 0.9 (3 Gi), CPU 0.5 (1.25 cores), from {0.5, 0.75, 0.9, 0.95}
rules	margins (tuned)	memory 1.5, CPU 1.0, GPU 1.0; first-run GPU quantile 0.5
rules-sized	cold start (tuned)	512 MiB + 4× input MiB; margin 1.5 with history
trees	quantile models	19 per axis, $\tau = 0.05 \ldots 0.95$; 300 rounds, lr 0.06, 31 leaves; $\theta = 0.3$
zero-shot	generation	27B Qwen (vLLM), temperature 0; 8,192 tokens free-form, or 256 tokens JSON-schema constrained; 16 concurrent requests
decision	torso	Qwen3-1.7B, LoRA r 16, α 32 on q, k, v, o, gate, up, down; prompt $\leq$ 3,072 tokens
decision	head	shared LayerNorm; per axis W_q, W_k: 2,048 $\rightarrow$ 256
decision	optimization	1 epoch over 27,936 rows (2 epochs over 13,968 for single-pod-type ablations); batch 8×2; AdamW 2×10^{-4} (LoRA) / 10^{-3} (head); 5% warm-up, cosine; CPU loss weight 0.3
decision	calibration	T = 1.09 / 1.04 / 1.04 (memory / CPU / GPU); $\theta = 0.3$
RL	GRPO	K = 16, 6,000 prompts, 1 epoch, batch 8×2, AdamW 3×10^{-5} / 10^{-4}, KL β 0.05, reward cap 20
RL	calibration	T = 5.0 / 5.0 / 5.0 (the search bound; seed 2's GPU: 4.13)
pools	w on trees (tuned)	× decision 0.5; × RL 0.6 (seed 3: 0.8); × Job-only 0.6 (seed 2: 0.4); × Flyte-only 0.4; $\theta = 0.3$
gate	signal	kNN, k = 10, Euclidean on train-standardized features; $q = 0.95$ of validation
cost	constants	0.02 h per attempt; 8 attempts; OOM and delay penalties 1; no-GPU failure 0.05 h

D Runs

Every cluster run behind this paper, in project `resource-tuner-model-factory` on `demo.hosted.unionai.cloud`. The three corpora were built in earlier rounds: RightSizeMem v2 (`rsb-v2-corpus-2`), RightSizeCPU big-1 (`rsc-big-1`) and RightSizeGPU big-2 (`rsg-big2-1`).

run	what
`rt-k8sprobe-1`	staging probe: boto3 footprint and throughput, tmpfs charging
`rt-k8sval-2`	176 runs as a Flyte task and as `job.py`, as sibling processes in one pod
`rt-k8spod-pilot-1..3`	real-pod probe pilots: LimitRange CPU limits, harness fixes, init-tmpfs failures found
`rt-k8spod-1`	real-pod probe: 100 runs, about 480 pods (Section 7)
`rsk-k8s-1`, `rsk-k8s-1-gate`	the first release (180 MiB runtime, flat init containers), two trainings
`rsk-k8s-2`	k8s-2, seed 1: every approach, the counterfactual and the name test
`rsk-k8s-2-s2`, `-s3`	k8s-2, seeds 2 and 3: every approach except zero-shot
`rt-paper-k2s*`	exports of every request per pod, for the offline analysis

E Supporting tables

Table 18: The difference between a run's Job pod and its Flyte pod, paired on the same runs (Job minus Flyte, in percentage points), for seed 1.

	Job − Flyte regret, paired (pp)	
approach	test	unseen
static	-2.2 [-3.2, -1.2]	+2.3 [+1.4, +3.4]
rules	+1.1 [-0.5, +2.5]	+1.7 [+0.8, +2.6]
rules-sized	+0.4 [-1.1, +1.6]	-1.8 [-2.5, -1.2]
zero-shot 27B	+13.1 [-4.8, +32.7]	-9.2 [-25.8, +6.7]
zero-shot 27B, JSON	+27.0 [+13.9, +41.5]	-7.1 [-37.9, +14.6]
trees	-3.1 [-4.0, -2.3]	+0.1 [-0.9, +1.1]
decision	+1.4 [+0.4, +2.3]	+4.8 [+3.5, +6.1]
decision, argmax	+1.6 [-0.0, +3.4]	+5.4 [+3.6, +7.0]
RL	+0.3 [-0.4, +1.0]	-0.9 [-2.0, +0.1]
RL, argmax	-0.1 [-1.0, +0.7]	+0.3 [-0.9, +1.5]
pool × decision	+1.1 [+0.5, +1.6]	+5.4 [+4.1, +6.8]
pool × RL	-0.6 [-1.2, +0.1]	-0.7 [-2.0, +0.5]
pool × RL, argmax	+0.1 [-0.8, +1.1]	+1.2 [-0.2, +2.6]
gate (kNN, q 0.95)	+0.6 [-0.4, +1.6]	-0.9 [-1.9, +0.1]
decision, Job-only	-7.3 [-8.6, -6.0]	+2.5 [+0.5, +4.4]
decision, Flyte-only	+25.3 [+23.1, +27.5]	+10.5 [+8.4, +12.6]

Table 19: Test pairs of a run's Flyte pod and its Job pod on CPU nodes (3,950 pairs, of which 29.9% have different labels). The first three columns give the share of moved pairs where the request moved the same way as the label; "unmoved, kept" gives the share of unmoved pairs where the request also stayed the same.

	label moved: direction right (%)				
approach	moved	up	down	unmoved, kept	all
rules	24.5	32.3	15.3	90.6	70.9
rules-sized	35.6	48.6	20.4	83.1	68.9
zero-shot 27B	53.8	38.5	69.2	30.4	36.8
trees	60.3	74.8	43.3	78.4	73.0
trees, Job-only	50.4	89.8	4.1	77.0	69.0
trees, Flyte-only	35.2	63.8	1.7	50.6	46.0
decision	63.6	81.2	42.9	78.9	74.3
decision, Job-only	57.3	64.7	48.6	69.2	65.6
decision, Flyte-only	43.3	61.4	21.9	58.8	54.2
RL	56.4	82.6	25.6	74.7	69.2
pool × decision	63.8	81.7	42.9	80.6	75.6
pool × RL, argmax	66.6	80.9	49.7	84.3	79.0
gate (kNN, q 0.95)	66.1	81.2	48.4	83.8	78.5

Table 20: Regret (%) on the counterfactual's first runs and their real Jobs, with the task's own name and with a neutral one, for seed 1. No model was retrained.

	test regret (%)			unseen regret (%)		
approach	named	neutral	Δ	named	neutral	Δ
static	83.0	83.0	+0.0	104.2	104.2	+0.0
rules	80.8	80.8	+0.0	82.7	82.7	+0.0
trees	17.9	18.6	+0.7	97.9	98.4	+0.5
decision	10.3	10.2	-0.1	83.7	86.5	+2.8
decision, Job-only	19.4	20.1	+0.7	82.4	83.6	+1.2
decision, Flyte-only	32.6	34.4	+1.8	92.4	93.9	+1.5
RL	11.7	12.3	+0.5	58.1	61.7	+3.6
pool × decision	6.4	7.2	+0.8	86.9	88.2	+1.4
pool × RL, argmax	12.9	13.4	+0.5	109.3	109.8	+0.5

Table 21: Job regret (%) on test CPU-node pods, by staging, for seed 1.

approach	init-disk	init-tmpfs	boto3-disk	boto3-tmpfs
static	85.2	79.1	60.5	69.6
rules	47.7	54.7	43.5	43.7
rules-sized	38.7	42.7	46.4	35.0
zero-shot 27B	42.3	67.7	26.5	75.6
zero-shot 27B, JSON	23.6	61.9	52.4	68.0
trees	12.8	13.8	8.9	5.3
decision	7.2	4.5	7.9	6.0
decision, argmax	30.3	25.4	27.3	22.0
RL	12.0	11.9	10.2	7.7
RL, argmax	5.9	3.7	**4.8**	3.4
pool × decision	**4.8**	**2.6**	5.8	**3.0**
pool × RL	10.7	10.4	6.5	4.7
pool × RL, argmax	9.3	7.0	6.7	3.7
gate (kNN, q 0.95)	9.5	7.7	6.9	4.6